Session 13 - Callback Functions

Naveen AutomationLabs | 2026 Playwright Batch G1

1. What is a Callback Function?

A callback is a function passed as an argument to another function. The receiving function calls it later. This is sometimes called 'call by function'.

- Call by value: passing a primitive value
- Call by object reference: passing an object
- Call by function (callback): passing a function itself

```
function sayHello(callback) {
callback();
}

sayHello(function () {
console.log('hiii');
});
```

2. Callback with Utility Functions (Calculator Pattern)

Define small utility functions and pass them as callbacks to a core function.

```
// Utility functions:
let add = (a, b) => a + b;
let sub = (a, b) => a - b;
let mul = (a, b) => a * b;
let div = (a, b) => a / b;

// Core function:
function calculator(a, b, callback) {
return callback(a, b);
}

console.log(calculator(10, 20, add)); // 30
console.log(calculator(100, 50, sub)); // 50
console.log(calculator(10, 30, mul)); // 300
console.log(calculator(10, 2, div)); // 5
```

3. Callback with Inline Anonymous/Arrow Functions

You can pass functions directly without defining them separately.

```
let t1 = calculator(10, 20, (a, b) => a + b);
console.log(t1); // 30

t1 = calculator(10, 20, (a, b) => a / b);
console.log(t1); // 0.5

// Using anonymous function syntax:
calculator(100, 200, function (a, b) {
```

```
return a + b;
});
```

4. Practical Example - Browser Launch with Callback

```
function initDriver(browserName) {
switch (browserName.trim().toLowerCase()) {
case 'chrome':
console.log('launch chrome');
return true;
case 'firefox':
console.log('launch firefox');
return true;
default:
console.log('wrong browser: ' + browserName);
return false;
}
}

function enterURL(browserName, url, callback) {
console.log('starting test execution.....');
if (callback(browserName)) {
console.log('enter the url:', url);
}
}

enterURL('chrome', 'https://www.nal.com', initDriver);
```

Same logic with inline arrow callback:

```
function launchBrowser(browserName, callback) {
return callback(browserName);
}

let flag = launchBrowser('chrome', (browserName) => {
switch (browserName.trim().toLowerCase()) {
case 'chrome':
console.log('launch chrome');
return true;
case 'firefox':
console.log('launch firefox');
return true;
default:
return false;
}
});

if (flag) {
console.log('enter the url');
}
```

5. Callbacks with Array Methods

Many built-in array methods use callbacks: forEach, filter, map, etc.

```
let num = [1, 2, 3, 4, 5];

// forEach with arrow function:
```

```
num.forEach(e => console.log(e + 5));

// forEach with anonymous function:
num.forEach(function (e) {
console.log(e + 5);
});

// filter with callback:
let filtered = num.filter(e => e > 3);
console.log(filtered); // [4, 5]
```

6. Passing Multiple Callbacks

```
function perform(callback1, callback2) {
callback1();
callback2();
}

perform(
() => console.log('hi'),
() => console.log('hello')
);
```

7. Callback with Rest Parameters

You can combine callbacks with rest parameters for flexible functions.

```
function calculator(mycallback, ...x) {
console.log(mycallback(...x));
}

let addition = (...a) => {
let sum = 0;
for (let i = 0; i < a.length; i++) {
sum = sum + a[i];
}
return sum;
};

calculator(addition, 1, 2, 3, 4); // 10
```

The callback (addition) receives all the rest parameters using the spread operator. This pattern lets you pass any number of values to the callback.